

CRYPTO HAWKING · ETHDENVER 2025 WINNER

Smart Contract Security Audit

SavingsDai

0x83F20F44975D03b1b09e64809B757c47f942BEeA

Ethereum

Compiler: v0.8.17+commit.8df45f5f

Date: October 7, 2026

OVERALL RISK RATING AI-assisted audit. Not a substitute for manual review.

Medium

Executive Summary

Independent security review of publicly verified source, not commissioned by the project. SavingsDai is the canonical MakerDAO sDAI wrapper implementing an ERC-4626-style vault over the DSR (Pot) contract. The core accounting logic (chi/rpow interest accrual, checks-effects-interactions ordering in mint/burn, EIP-2612/1271 permit handling) is sound and mirrors the well-audited production contract. Findings are limited to missing ERC20 Transfer events on mint/burn (affecting off-chain indexers), unchecked return values on trusted external token calls, and minor best-practice/style issues. No critical or high-severity fund-loss vectors were identified given the immutable, non-upgradeable, ownerless design.

Findings (4)

F-001

Medium

Mint/Burn do not emit standard ERC20 Transfer events

SavingsDai.sol:214–248

DESCRIPTION

The `_mint` and `_burn` internal functions update `balanceOf` and `totalSupply` and emit custom `Deposit`/`Withdraw` events, but never emit the standard ERC20 `Transfer(address(0), receiver, shares)` / `Transfer(owner, address(0), shares)` events that most wallets, block explorers, DEX aggregators, and accounting tools rely on to track balance changes.

IMPACT

Off-chain indexers, portfolio trackers, and any integration relying solely on `Transfer` events (rather than `Deposit`/`Withdraw`) will fail to detect sDAI minted or burned via `deposit`/`mint`/`withdraw`/`redeem`, leading to incorrect balance/supply reporting and potential accounting discrepancies in downstream systems.

RECOMMENDATION

Emit `Transfer(address(0), receiver, shares)` inside `_mint` and `Transfer(owner, address(0), shares)` inside `_burn` in addition to the existing `Deposit`/`Withdraw` events, matching standard ERC20/ERC4626 conventions.

```
unchecked {
  balanceOf[receiver] = balanceOf[receiver] + shares;
  totalSupply = totalSupply + shares;
}
emit Deposit(msg.sender, receiver, assets, shares);
// Missing: emit Transfer(address(0), receiver, shares);
```

F-002 · SWC-104

Low

Unchecked return values on external DAI token calls

SavingsDai.sol:96–216

DESCRIPTION

`dai.approve(address(daiJoin), type(uint256).max)` in the constructor and `dai.transferFrom(msg.sender, address(this), assets)` in `_mint` do not check the boolean return value of these ERC20 calls.

IMPACT

If the configured `dai` token ever deviated from standard Dai semantics (e.g., returning false instead of reverting on failure), a failed transfer/approval could silently proceed, leading to inconsistent internal accounting (shares minted without backing assets actually received). Risk is low in practice because MCD Dai always reverts on failure, but this is not guaranteed for any future redeployment or token substitution.

RECOMMENDATION

Wrap external token calls with explicit success checks, e.g. `require(dai.transferFrom(msg.sender, address(this), assets), "SavingsDai/transfer-failed");`, or use a SafeERC20-style wrapper for defense in depth.

```
dai.transferFrom(msg.sender, address(this), assets);
daiJoin.join(address(this), assets);
pot.join(shares);
```

F-003 · SWC-103

Low

Floating pragma

SavingsDai.sol:21–undefined

DESCRIPTION

The contract uses a floating pragma `^0.8.17`, allowing compilation with any compatible 0.8.x compiler version greater than or equal to 0.8.17.

IMPACT

Deploying with an unintended/untested compiler version could introduce subtle semantic changes or newly discovered compiler bugs, affecting bytecode reproducibility and audit guarantees.

RECOMMENDATION

Lock the pragma to the exact audited compiler version, e.g. `pragma solidity 0.8.17;`.

```
pragma solidity ^0.8.17;
```

F-004 · SWC-117

Informational

ECDSA signature malleability not restricted in permit

SavingsDai.sol:313–332

DESCRIPTION

`\_isValidSignature` accepts any `(v, r, s)` triple that recovers to the signer via `erecover` without constraining `s` to the lower half of the curve order, as recommended by EIP-2 / OpenZeppelin's ECDSA library. Since the permit nonce is incremented per call, malleable signatures cannot be replayed to re-spend an allowance, but a third party could front-run and submit an alternate valid signature for the same intended permit message using a flipped `s`/`v` pair before the original transaction lands, consuming the nonce with different transaction metadata (not economically exploitable here, but non-standard).

IMPACT

No direct fund loss given nonce-based replay protection, but deviates from best-practice ECDSA validation and could interact poorly with future contract changes.

RECOMMENDATION

Restrict `s` to `uint256(s) <=

0x7FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF5D576E7357A4501DDFE92F46681B20A0` before calling `ecrecover`, or use OpenZeppelin's `ECDSA.recover` which enforces this check.

Gas Optimizations

Duplicate chi computation logic across view functions

SavingsDai.sol:264–332

The pattern `(block.timestamp > rho) ? _rpow(pot.dsr(), block.timestamp - rho) * pot.chi() / RAY : pot.chi()` is repeated in `convertToShares`, `convertToAssets`, `previewMint`, `previewWithdraw`. Extract into a single internal `_chi()` internal view returns (uint256) helper to reduce bytecode size and improve maintainability.

Redundant external calls to pot.rho()/pot.chi()/pot.dsr()

SavingsDai.sol:264–300

Each preview/convert function makes up to 3 separate external calls to the `pot` contract. Where multiple values are needed in the same call, consider batching reads or caching where semantically safe to reduce external call overhead.

Appendix - Metadata

Contract	SavingsDai
Address	0x83F20F44975D03b1b09e64809B757c47f942BEeA
Chain	Ethereum
Compiler	v0.8.17+commit.8df45f5f
Audit ID	portfolio-spark-sdai
Date	10/7/2026

Files analyzed

[user] SavingsDai.sol

MANUAL AUDIT UPGRADE

Need a deeper audit?

\$5,000

Pay in ETH, SOL, or USDT. Detailed manual review by Crypto Hawking, ETHDenver 2025 Winner. Comprehensive findings, remediation steps, and post-fix re-review. Report delivered in 3 business days.

[Request Manual Audit '](https://cryptohawking.com/audit/manual?ref=portfolio-spark-sdai)

<https://cryptohawking.com/audit/manual?ref=portfolio-spark-sdai>

Need it built, not just audited?

I also build production smart contracts end to end, token launches, DeFi protocols, staking, and full DEX stacks (v2, v3 concentrated liquidity, and beyond), plus fractional CTO engagements for Web3 teams. Fixes for the findings in this report can be scoped in the same engagement.

<https://cryptohawking.com/services>

WhatsApp (preferred): +92 322 4274236, <https://wa.me/923224274236>